KCA UNIVERSITY

OOT ASSIGNMENT ONE

Student Name: JOSEPH NDUNG'U
Registration No: 24/00727
Unit: Object-Oriented Technologies

A) CLASSES, ATTRIBUTES, AND OPERATIONS[SPAN_0](START_SPAN)[SPAN_0](END_SPAN)

1. University[span_1](start_span)[span_1](end_span)

- **Attributes:** name: String, address: String, registrationNumber: String[span_2](start_span)[span_2](end_span)
- **Operations:** addDepartment(dept: Department): void, getDepartmentList(): List<Department>[span_3](start_span)[span_3](end_span)

2. Department[span_4](start_span)[span_4](end_span)

- **Attributes:** departmentId: String, deptName: String, officeLocation: String[span_5](start_span)[span_5](end_span)
- **Operations:** addCourse(course: Course): void, assignLecturer(lecturer: Lecturer): void[span_6](start_span)[span_6](end_span)

3. Lecturer[span_7](start_span)[span_7](end_span)

- **Attributes:** lecturerId: String, name: String, email: String[span_8](start_span)[span_8](end_span)
- **Operations:** assignCourseCoordination(course: Course): void, submitGrade(studentId: String, courseId: String, grade: String): void[span_9](start_span)[span_9](end_span)

4. Course[span_10](start_span)[span_10](end_span)

- **Attributes:** courseCode: String, title: String, creditUnits: int[span_11](start_span)[span_11](end_span)
- **Operations:** getEnrolledStudents(): List<Student>, setCoordinator(lecturer: Lecturer): void[span_12](start_span)[span_12](end_span)

5. Student (Abstract)[span_13](start_span)[span_13](end_span)

- **Attributes:** studentId: String, fullName: String, email: String[span_14](start_span)[span_14](end_span)
- **Operations:** registerCourse(course: Course, semester: String): Registration, viewTranscript(): void[span_15](start_span)[span_15](end_span)

6. UndergraduateStudent[span_16](start_span)[span_16](end_span)

- **Attributes:** highSchoolCertNo: String, attachmentStatus: Boolean, maxUndergradCredits: int[span_17](start_span)[span_17](end_span)
- **Operations:** checkGraduationRequirement(): Boolean, requestIndustrialAttachment(): void[span_18](start_span)[span_18](end_span)

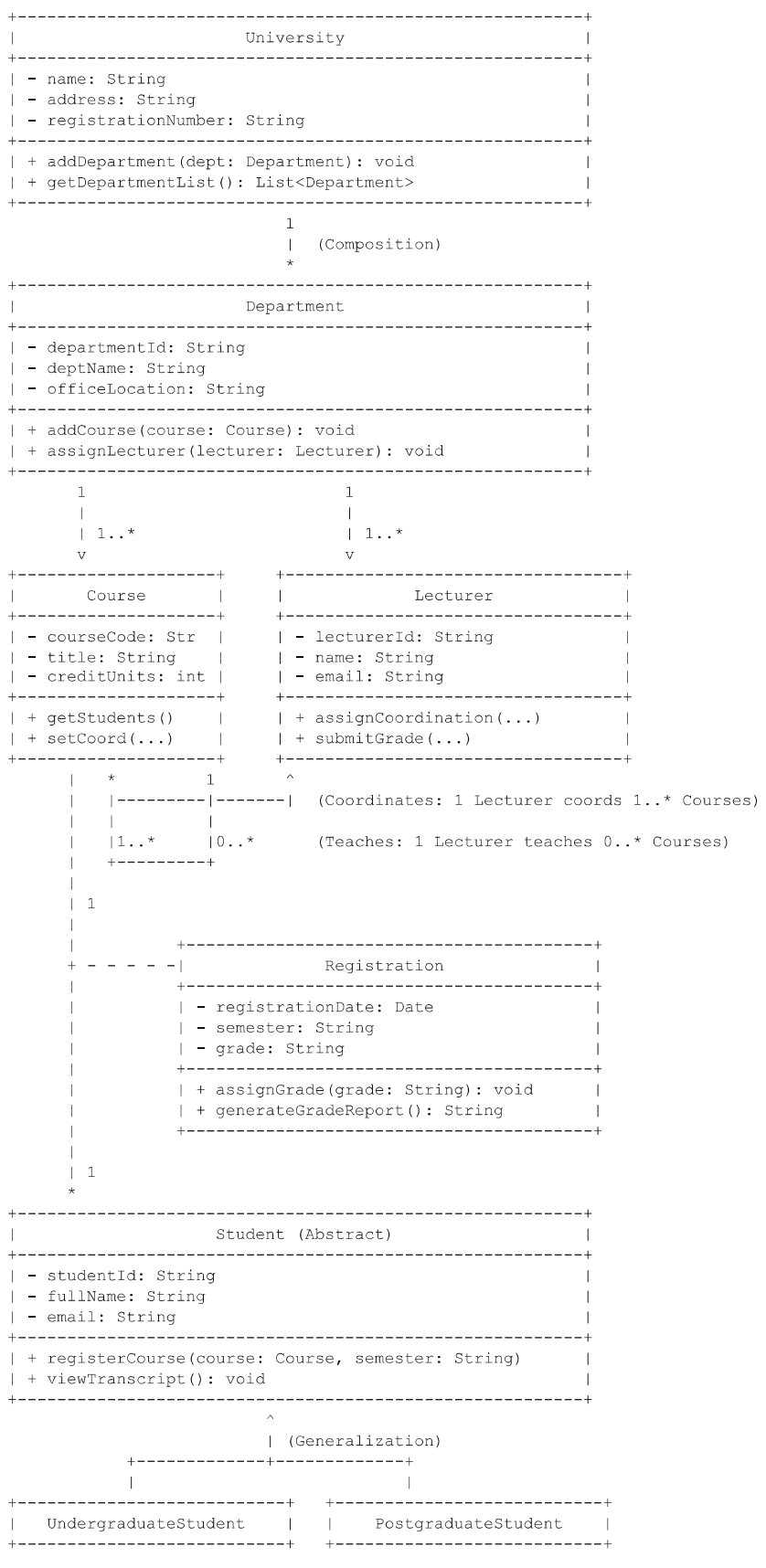
7. PostgraduateStudent[span_19](start_span)[span_19](end_span)

8. Registration[span_22](start_span)[span_22](end_span)

- **Attributes:** researchTopic: String, supervisorName: String, hasPublishedPaper: Boolean[span_20](start_span)[span_20](end_span)
- **Operations:** submitThesisProposal(): void, checkDefenseEligibility(): Boolean[span_21](start_span)[span_21](end_span)

- **Attributes:** registrationDate: Date, semester: String, grade: String[span_23](start_span)[span_23](end_span)
- **Operations:** assignGrade(grade: String): void, generateGradeReport(): String[span_24](start_span)[span_24](end_span)

B) UML CLASS DIAGRAM[SPAN_25](START_SPAN)[SPAN_25](END_SPAN)



```
| - highSchoolCertNo: String|   | - researchTopic: String   |
| - attachmentStatus: Bool  |   | - supervisorName: String  |
| - maxUndergradCredits: int|   | - hasPublishedPaper: Bool |
+---------------------------+   +---------------------------+
| + checkGraduationReq()    |   | + submitThesisProposal()  |
| + requestAttachment()     |   | + checkDefenseEligibility |
+---------------------------+   +---------------------------+
```

C) OOP RELATIONSHIPS JUSTIFICATION[SPAN_26](START_SPAN)[SPAN_26](END_SPAN)

- **Inheritance (Generalization):** Applied between `Student` and subclasses `UndergraduateStudent` and `PostgraduateStudent`[span_27](start_span)[span_27](end_span). Both types share common properties (ID, name, course registration), but have different academic rules (thesis vs. industrial attachments)[span_28](start_span)[span_28](end_span). This avoids code duplication and enables polymorphism[span_29](start_span)[span_29](end_span).
- **Composition:** Applied between `University` and `Department`[span_30](start_span)[span_30](end_span). A department cannot exist without the university; if the university ceases to exist, its departments are dissolved[span_31](start_span)[span_31](end_span).
- **Association:** Applied between `Department` and `Course` / `Lecturer`, and between `Lecturer` and `Course`[span_32](start_span)[span_32](end_span). These entities interact and reference one another without strictly binding their lifecycles (e.g., a lecturer departing does not destroy the course or department)[span_33](start_span)[span_33](end_span).

D) RESOLVING THE MANY-TO-MANY RELATIONSHIP[SPAN_34](START_SPAN)[SPAN_34](END_SPAN)

A direct many-to-many relationship (`Student ↔ Course`) cannot store enrollment-specific attributes like **registration date**, **semester**, and **grade**[span_35](start_span)[span_35](end_span). These attributes belong neither exclusively to a student nor to a course, but to the specific enrollment event[span_36](start_span)[span_36](end_span).

Introducing the associative class `Registration` decomposes the complex $* \leftrightarrow *$ link into two clean $1 \leftrightarrow *$ relationships[span_37](start_span)[span_37](end_span):

Student (1) —— (*) Registration (*) —— (1) Course[span_38](start_span)[span_38](end_span)

This structure prevents multi-valued attributes, normalizes the data model, and allows students to re-attempt courses in different semesters while preserving historical transcripts[span_39](start_span)[span_39](end_span).